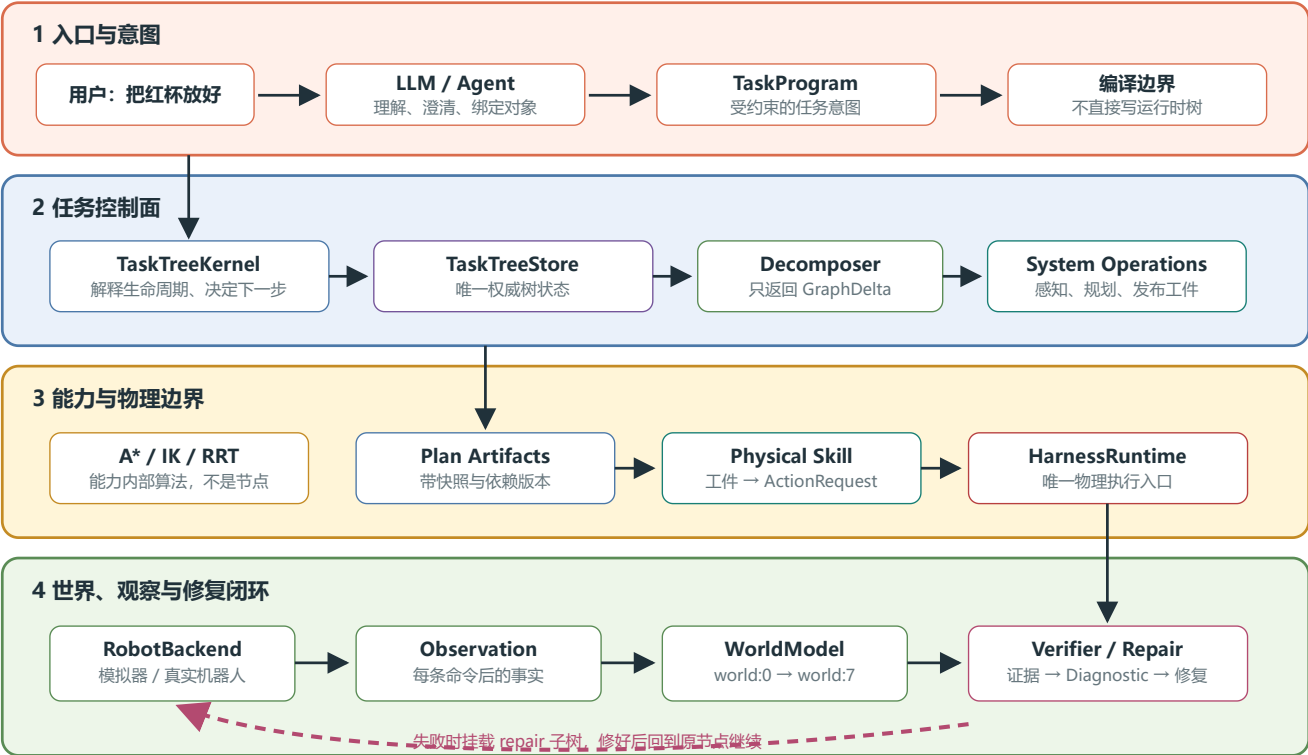


把 Agent 框架 讲到人人都能看懂

从一句“把红杯放好”，一路讲到任务树、规划工件、机器人动作、观察证据，以及失败后的递归修复。

白话 结构图 真实案例 故障修复



先记住一句话： LLM 负责理解目标，Kernel 负责控制过程，Harness 负责承载真实世界，Repair 负责把失败重新变成工作。

先建立一张心智地图：这套系统到底在做什么？

先把下面这条主线记住，后面每个技术名词都会回到它。

一句人话

它不是“让大模型直接控制机器人”，而是给 Agent 加了一层可检查、可追踪、可修复的执行控制面。

1 理解 把人话变成明确目标	2 拆解 把目标变成任务树	3 执行 把计划变成受控动作	4 恢复 发现失败就挂修复树
---	--	---	---

你先要建立的直觉

- 出了问题能说清楚：哪一步、为什么、影响什么。
- 不是每次失败都人工从头开始，能局部修复。
- 换算法、换机器人时，业务任务不用全部重写。
- 连续任务能保持真实现场，不会每次都把世界重置。

你随后要看懂的机制

- TaskTreeStore 是唯一权威运行时树状态。
- TaskTreeKernel 是唯一节点生命周期解释器和 Store 写入者。
- LLM 只能提交规范化 TaskProgram，不能写树或调机器人。
- HarnessRuntime 是唯一物理执行入口，每条命令后都有 Observation。

读图方法

后面每一页都回答一个具体问题：谁负责？看什么证据？失败后怎么办？不要把“模型会说话”误认为“系统已经可靠执行”。

第一段 第 3-7 页：先用杯子任务建立直觉	第二段 第 8-20 页：看清对象、分层、权限和生命周期	第三段 第 21-26 页：完整走一遍正常任务	第四段 第 27-32 页：完整走一遍故障与递归修复	第五段 第 33-41 页：连续会话、安全、边界和总结；第 42 页以后继续看对象编码与运行时细节
--	--	---	--	---

先别怕术语：我们真正要解决的是什么？

先用一个生活里最简单的动作，把所有复杂词汇压缩成一句话。

“请把红色杯子放到右边的投放区。”

这句话听起来只有一个动作，但机器人要面对一串现实问题。

对象是谁？

识别

红色杯子是哪个实体？现场可能有多个杯子，不能只凭名字猜。

目标在哪里？

绑定

“右边”是哪个区域？需要一个可验证的目标位置和坐标。

路线能走吗？

规划

中间有没有墙、箱子或其他障碍？拿起杯子后路线是否仍然成立？

动作真的成功吗？

验证

命令发出不等于杯子被拿起，更不等于杯子最终已经放好。

核心转折

人只说“结果”，系统必须把结果拆成一连串可检查的前置条件、动作和后置证据。

为什么不能让 LLM 直接调用机器人？

直接调用很快能跑通 Demo，但在真实世界里，最危险的恰恰是“看起来跑通了”。

黑盒方式

LLM → robot.move() → 返回 success

- 模型既决定做什么，又决定怎么做，还可能直接触发副作用。
- 失败只剩一句“没成功”，很难定位到底是路线、抓取还是验证出了问题。
- 重试可能重复抓取、重复释放，造成二次副作用。
- 过程没有统一的任务状态、快照、事务和审计边界。

控制面方式

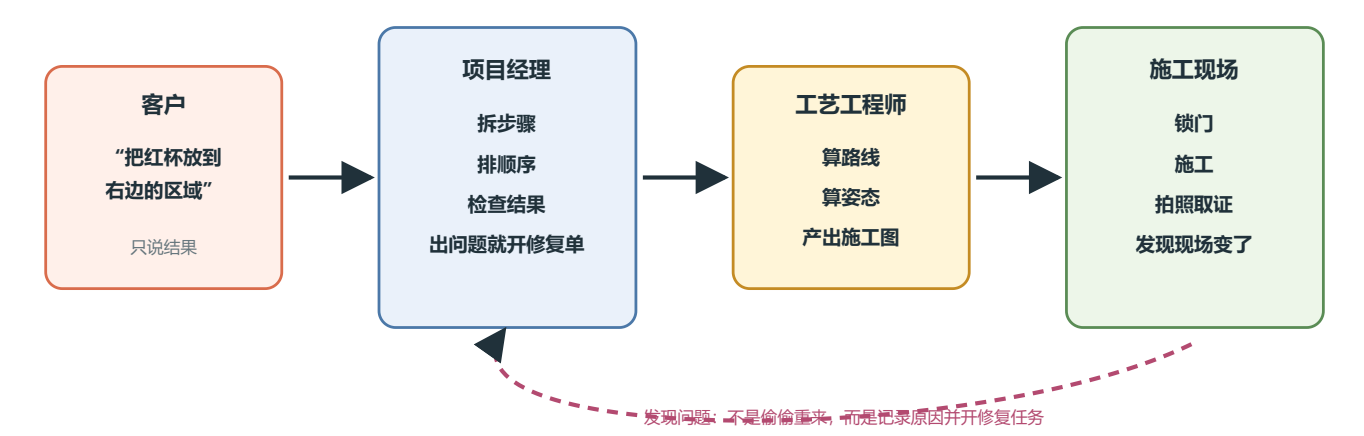
意图 → 任务树 → 工件 → ActionRequest → Runtime → Observation

- 每一步都有明确责任人和权限。
- 每次动作都能关联到任务节点、计划版本和世界快照。
- 失败先变成结构化 Diagnostic，再决定是否修复。
- 模型不能越过 Runtime 直接碰后端。

关键区别： 黑盒关注“有没有返回值”；控制面关注“现场是否真的变了，而且变成了预期的样子”。

用一座工厂来理解整套架构

这是最容易记住的类比：用户是客户，LLM 是接单员，Kernel 是项目经理，规划器是工艺工程师，Runtime 是施工总闸门。



客户	只说想要的结果：“杯子放好”。
接单员 / LLM	理解人话，确认对象，填写标准订单。
项目经理 / Kernel	安排顺序，记录状态，决定失败后下一步。
工艺工程师 / A*、IK、RRT	计算路线、姿态和运动方案，但不偷偷搬东西。
施工总闸门 / HarnessRuntime	检查现场、锁资源、执行动作、收集证据。

为什么这个类比重要？

因为它把“会思考”和“有权执行”分开了。项目经理可以决定下一步，但不应该绕过安全闸门直接开机器。

一句人话，如何变成系统能执行的任务？

系统不会把整句自然语言原样塞给机器人，而是先转换成一个受约束的任务意图。



人说的	系统必须明确的	为什么
红杯	实体 ID: cup-red	防止抓错对象
投放区	区域 ID: drop-zone	防止目标漂移
放好	位置误差、释放状态、最终验证	防止“命令成功但结果不对”

不要把它理解成形式主义

这些字段的作用是把模糊的人话变成可以检查、可以追责、可以重试的对象。

最小案例：Place 到底包含多少事情？

用户只说一个“放置”，系统内部至少要回答五类问题。

目标	把 cup-red 放到 drop-zone
准备	找到对象、确认目标、生成抓取计划
移动	先到抓取位置，再拿起，再带着物体走
完成	释放、确认已松开、确认最终位置正确
异常	路线被挡时，先修复阻碍，再回到原任务



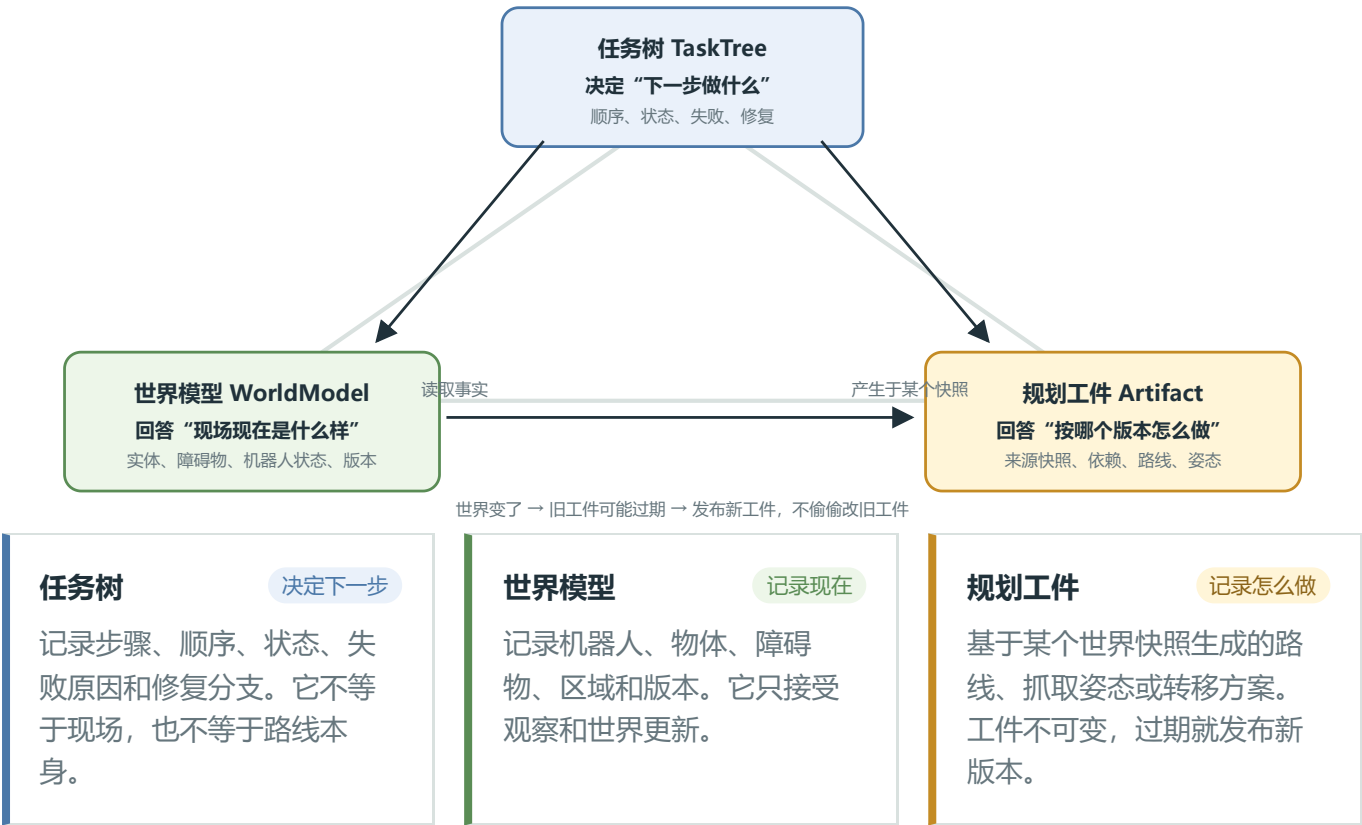
注意：这里的“+”不是数学加法，而是说明一个大目标由多个有先后关系的小工作组成。每个小工作都能独立记录成功或失败。

提前规划的原因

系统会在还没有拿起杯子时，先检查“带着杯子能不能走到目标”。如果这时就发现路不通，修复会更安全，因为夹爪还是空的。

整套架构只有三个核心对象要先记住

把这三样东西分开，后面的所有设计就不容易混乱。



最简单的记忆口诀
树管顺序，世界管事实，工件管方案。

什么是任务树？ 把它当成 “会执行的待办清单”

普通待办清单只写要做什么；任务树还记录顺序、状态、证据和失败后的分支。



任务树不是日志，也不是一大坨 JSON

这三个东西各有职责，混在一起以后，谁拥有状态、哪个计划过期、失败在哪里都会变得不清楚。

任务树

结构

“应该按什么顺序做？现在做到哪一步？”

由 Kernel 解释，由 Store 保存。

世界快照

事实

“机器人和物体此刻在哪里？现场版本是多少？”

由 Observation 推进。

规划工件

方案

“在 world:0 的条件下，路线和姿态怎么走？”

不可变，按依赖判断新鲜度。

错误想象

```
{ intent, current_state, plan, result, error, retry }
```

所有东西挤在一个对象里，看似省事，实际上无法准确判断“计划为什么失效”。

正确的分工

任务树可以继续存在，世界快照可以继续推进，旧工件可以保留作审计；三者不需要互相覆盖。

什么是 WorldModel? 它就是系统的“现场账本”

机器人不是生活在模型的想象里，而是生活在一个会变化的物理世界里。

world:7 · 当前世界快照

机器人	base=(6.5, 4.5), 夹爪已打开, 未持物	cup-red	位于 drop-zone 中, 实体版本 4
drop-zone	目标区域, 允许误差 0.3	map	障碍占用版本已记录



“世界版本”不是为了让数字好看，而是为了回答：这份计划是根据哪一刻的现场算出来的？在它之后现场有没有发生过会影响计划的变化？

最重要的边界

规划器和 Verifier 读取 WorldSnapshot；它们不能直接读取机器人后端的私有状态，更不能绕过观察把“猜测”写成事实。

什么是 Artifact? 把它当成 “带版本的施工图”

规划算法算出来的不是一句 “我觉得能走”，而是一份带来源、依赖和使用条件的不可变工件。

TransferPlan		不可变
来源快照	world:0	
依赖版本	drop-zone=0 · map=0 · robot=0	
路线内容	起点姿态 → 中间路径点 → 目标姿态	
安全信息	占用指纹、机器人模型、载荷约束、校验策略	
消费者	TransferHeld physical skill	



旧施工图不能被偷偷涂改

如果现场变了，就保留 v1，重新生成 v2，并记录为什么换图。

- 保留旧工件**

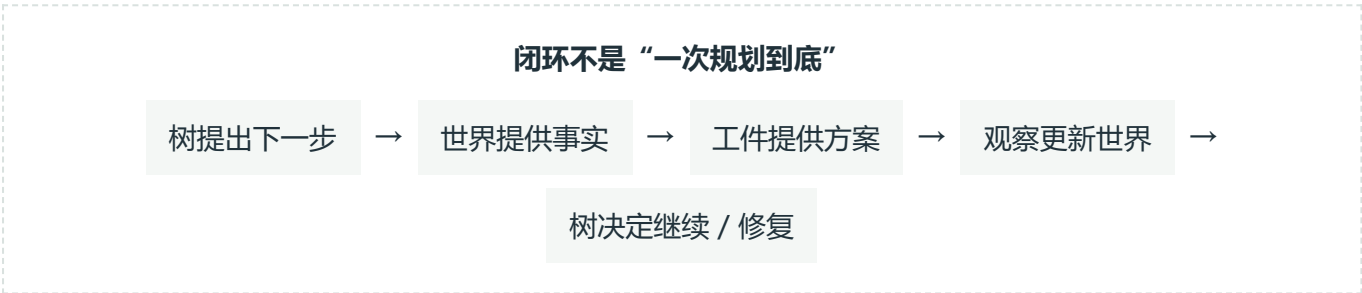
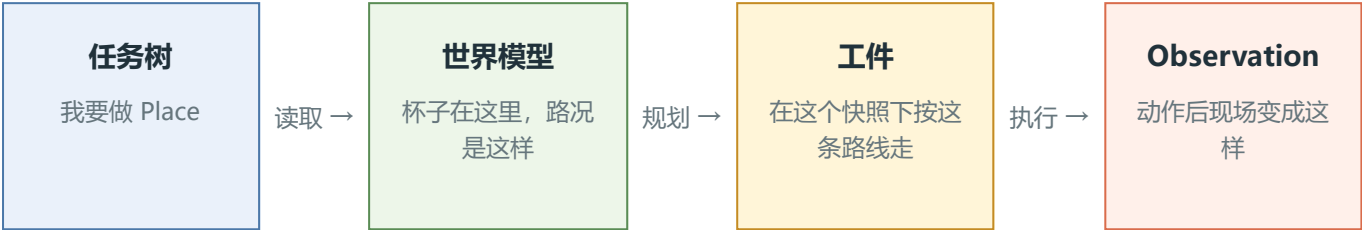
便于审计、回放和解释。
- 发布新工件**

让后续动作使用与当前现场匹配的方案。
- 按依赖判断新鲜度**

不是随意设置一个 “过期” 布尔值。

三个核心对象如何连起来？

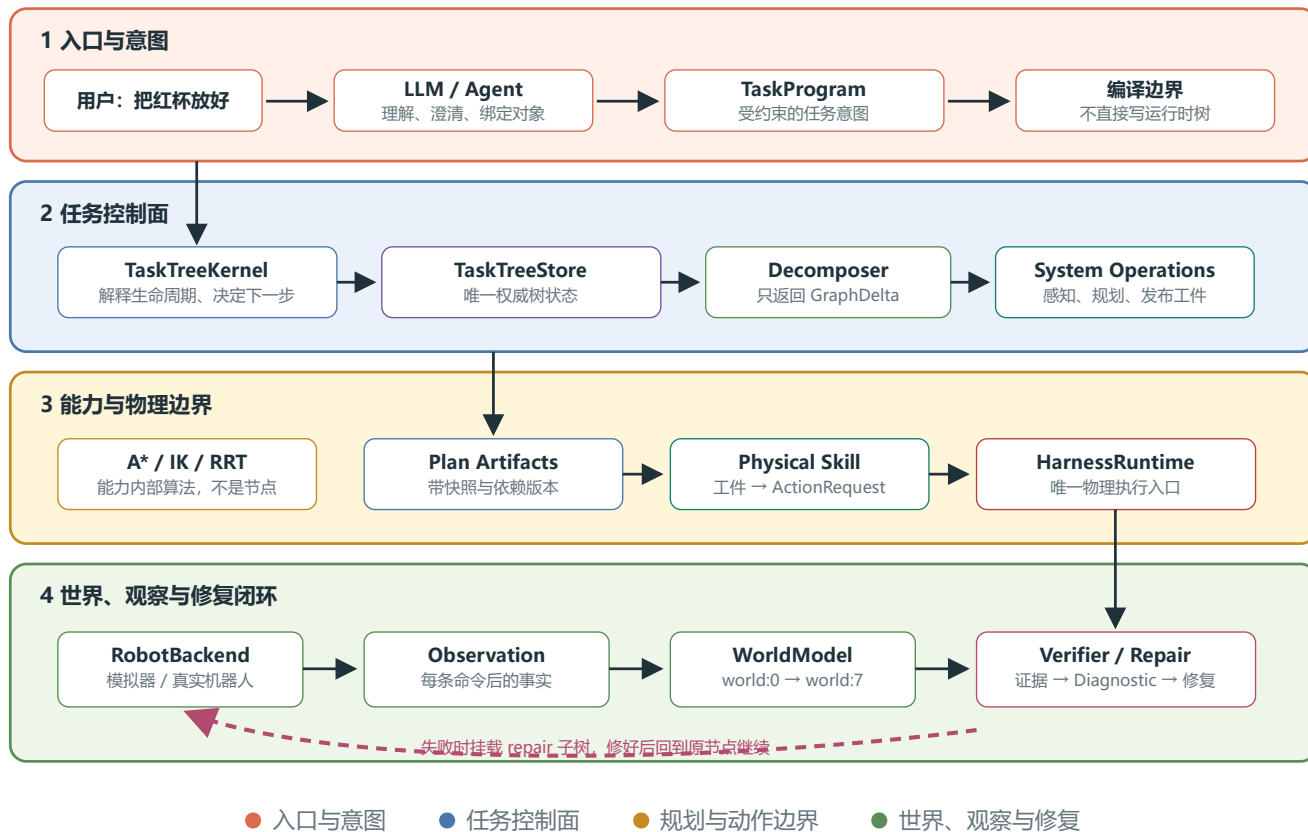
系统不是先凭空写一棵大树，再把它强行执行；而是让树、世界和工件互相校验。



用一句话说
树决定“做什么”，工件说明“怎么做”，观察证明“做完后真的发生了什么”。

全局架构图：从输入一直走到物理世界

下面这张图是整套系统的“地图”。先看四层，再看箭头，不要一上来背模块名。



看箭头的顺序

实线是正常推进：输入 → 编译 → Kernel → 工件 → Runtime → 机器人；虚线是反馈：Observation → WorldModel → 验证 / 修复 → 回到 Kernel。

入口层负责“收任务”，不负责“偷偷做动作”

浏览器、语音、HTTP 和连续会话都只是入口适配器，它们不拥有运行时树的写权限。



入口可以做

- 提交对象和目标标识。
- 读取世界、树、事件和历史。
- 请求完整场景重置。
- 显示任务进度和最终结果。

入口不能做

- 直接修改运行时节点。
- 直接挂载 repair 子树。
- 直接调用规划器或机器人后端。
- 把多个根目标硬塞进同一棵树。

当前实现的真实边界

连续会话目前是串行执行：每个任务有自己的 Store / Kernel / 树，但共享同一个 WorldModel、Runtime、事务账本和机器人后端。

LLM 能做什么，不能做什么？

这不是削弱模型，而是把模型放在它最擅长、也最容易被约束的位置。

LLM 可以做

- 理解自然语言目标。
- 向用户追问缺失信息。
- 把对象和目标绑定成明确标识。
- 提交规范化的 TaskProgram。
- 提出高层任务意图。

LLM 不可以做

- 直接修改 TaskTreeStore。
- 直接调用 RobotBackend。
- 伪造 Observation 或验证结果。
- 跳过 Runtime 的安全、租约和事务。
- 把内部算法迭代伪装成业务节点。

权限

模型提交“申请单”，系统决定“能不能执行”。

如果模型给出了一个不合法任务，系统应该拒绝或要求澄清，而不是因为“模型很聪明”就放宽边界。可靠性来自约束，不来自祈祷。

编译器和 GraphDelta：为什么不允许随便改树？

分解器不是拿到一把“改数据库”的钥匙，而是提交一份受约束的结构增量。



- 只能把 child 挂在当前展开节点下面。
- repair 必须恰好从失败节点挂一条 repair 边。
- 新节点不能跑出声明的子树范围。
- 完整图必须无环、可达、可解释。

为什么叫 Delta？
因为它只描述“增加什么结构”，而不是让外部组件直接改掉整棵树。这样 Kernel 仍然是唯一的解释者和写入者。

Kernel 与 Store：一个负责解释，一个负责保存

这是整个架构最重要的所有权边界。谁能写状态，决定了系统能不能被控制。



如果每个模块都能写树，会发生什么？

一个模块标记 succeeded，另一个模块又改成 failed；修复分支没有父节点；执行记录 and 实际状态互相打架。

三类节点：分解、系统、物理，各自只能拿到一部分权限

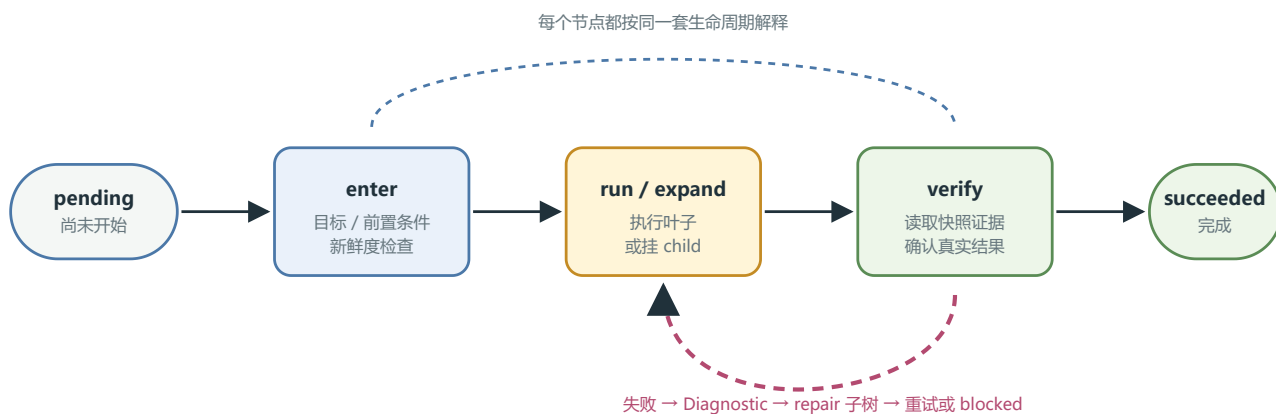
节点类型不是名字游戏，而是“它能看到什么、能改变什么”的运行时契约。

D Decomposer 把一个复合任务拆成 child 节点。 能做： 返回 GraphDelta 不能做： 调机器人、直接写 Store	S System Operation 读世界、做选择、做规划、发布工件、做验证。 能做： 读 WorldSnapshot、调用能力 不能做： 直接触碰 Backend	P Physical Skill 把已经批准的工件翻译成动作请求。 能做： 读声明的 Artifact 不能做： 绕过 Runtime 执行
--	---	--

A*、IK、RRT 放在哪里？	它们是能力内部的算法。算法迭代次数不应该变成任务树节点，否则树会记录大量没有业务意义的内部细节。
---------------------------------------	---

一个节点从 pending 到 succeeded, 究竟经历了什么?

所有节点都经过同一个 Kernel 生命周期。复合节点可以展开，叶子节点可以执行，但最后都要验证。



进入

读取目标和当前状态，检查是否已经满足。

前置

确认工件新鲜、条件成立、资源可用。

执行 / 展开

叶子做工作，复合节点返回 GraphDelta。

后置

根据不可变快照和证据判断结果。

失败

生成 Diagnostic，可能挂 repair，也可能进入 blocked。

一个容易误解的细节

某个目标已经满足时，叶子节点可以跳过动作；但结构节点不能因为目标满足就跳过必要的展开，否则树形结构和后置验证会失去意义。

案例一：正常放置任务的完整任务递归树

这是“没有故障”的参考执行。树里记录的是业务步骤，不是 A* 或控制器的每一轮计算。

16 最终节点数 1 个根 + 15 条 child 边对应的节点关系	15 child 边 正常分解关系	0 repair 边 本次没有故障修复	5 物理事务 5 个物理节点	127 事件数 可回放的执行证据
--	--	--	---	---



读这棵树

先看根节点 Place，再按从上到下的顺序读。缩进表示子任务；绿色 succeeded 表示后置验证已经通过，而不只是“函数返回了”。

案例一：正常任务按步骤走一遍

现在不要看代码，只把它当成一个现场工作人员的操作流程。

1

ResolveAndInspect

确认 cup-red 和 drop-zone 确实存在，建立对象绑定。

产出 BindingArtifact

2

PlanPick

规划机器人如何到杯子旁边，以及如何抓取。

产出 NavigationPlan + PickPlan

3

PlanTransfer

在还没拿杯子时，先规划带着杯子去目标区的路线。

产出 TransferPlan

4

Pick

到抓取位置 → 执行抓取 → 观察确认杯子确实被夹住。

3 个子节点

5

TransferHeld

切换携物姿态 → 带着杯子走 → 确认可以释放。

3 个子节点

6

Release

执行释放 → 观察确认夹爪松开且杯子不再被持有。

2 个子节点

7

VerifyPlaceGoal

检查杯子最终位置落在 drop-zone 接受范围内。

最终结论由证据得出

案例一：为什么一次成功不是 “一次 API 调用成功” ？

参考数据里，一次正常 Place 任务有 5 次物理事务、127 个事件，并把世界从 world:0 推进到 world:7。

world:0 起始现场	T1 NavigateToPickStance 观察后 world:1	T2 ExecuteGrasp 观察后 world:2	T3 MoveToTransportPosture 观察后 world:3	T4 NavigateHeld 观察后 world:4	T5 ExecuteRelease 观察后 world:7	world:7 目标已达成
------------------------	--	--	--	--	--	-------------------------

事件 127 包括进栈、状态变化、GraphDelta、出栈等	物理事务 5 每个事务都有请求 ID、检查、执行、观察	节点 16 业务可读、可定位、可回放
---	---	--

为什么事件比物理事务多？

因为一个物理动作前后还要记录节点进入、状态变化、执行栈和观察处理。多出的记录不是噪音，而是为了知道 “系统是怎样走到这个结果的” 。

一个物理动作，要过六道门才算完成

这六道门把“计划”变成“受控副作用”。任何一门不通过，都不能假装动作已经发生。



物理技能只负责构造 `ActionRequest`；真正的检查、租约、事务、执行、观察和对账由 `HarnessRuntime` 统一完成。

为什么每条命令后都必须有 Observation?

因为命令是“请求”，观察才是“事实”。系统不能把两者混为一谈。



为什么计划会过期？ 因为世界会变

同一条路线在 world:0 可行，不代表到了 world:7 还应该照着执行。



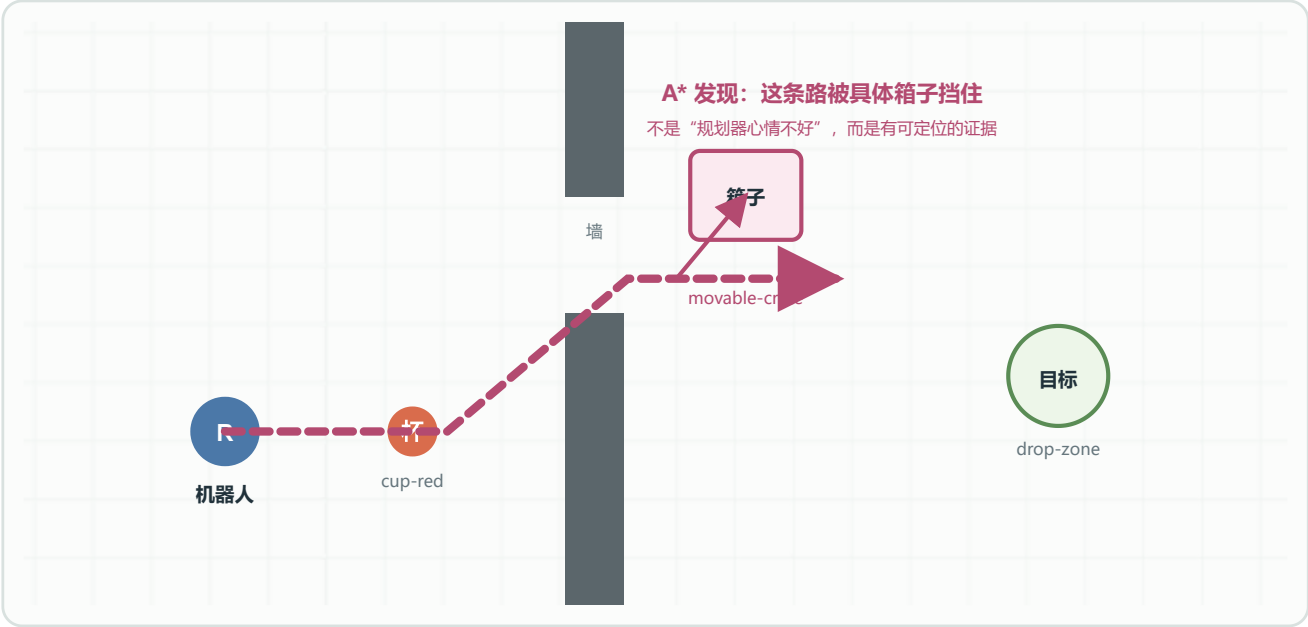
工件记录	当前世界比较什么	不一致意味着
source snapshot	快照引用	不能把旧现场当新现场
entity / map version	实体和地图版本	障碍或目标可能变化
robot state epoch	机器人状态纪元	轨迹起点可能不再连续
model / policy fingerprint	模型与安全策略	原计划不满足当前规则

不可变的意义

旧工件仍然可以被审计；新工件负责继续执行。系统不是把历史擦掉，而是把“为什么换计划”留下来。

案例二：路线被箱子挡住了

故障不是“机器人突然不聪明”，而是规划器发现了一个可以描述、可以验证的现实条件。



故障类型	ROUTE_BLOCKED	阻碍物	movable-crate，可移动，且被 A* 识别为具体 witness
修复前提	移走这个箱子后，反事实 A* 能证明路线恢复；并且存在可达 parking-zone。	当前状态	还没有拿起杯子，夹爪是空的，因此可以安全先处理箱子。

为什么不能简单地“再试一次”？

如果现场条件没有改变，重试只会重复同一个失败；如果现场已经部分改变，盲目重试还可能造成二次副作用。

做法	表面上发生什么	实际风险
原计划再跑一次	继续撞同一条被挡的路线	没有改变失败原因
让 A* 偷偷搬箱子	路线算法内部产生物理副作用	任务树看不到搬箱子，无法审计和验证
让模型自由发挥	可能临时调用未授权动作	越过安全边界，难以回放
结构化修复	先挂 repair 子树搬箱子，再刷新依赖工件	原因、动作、结果都可追踪

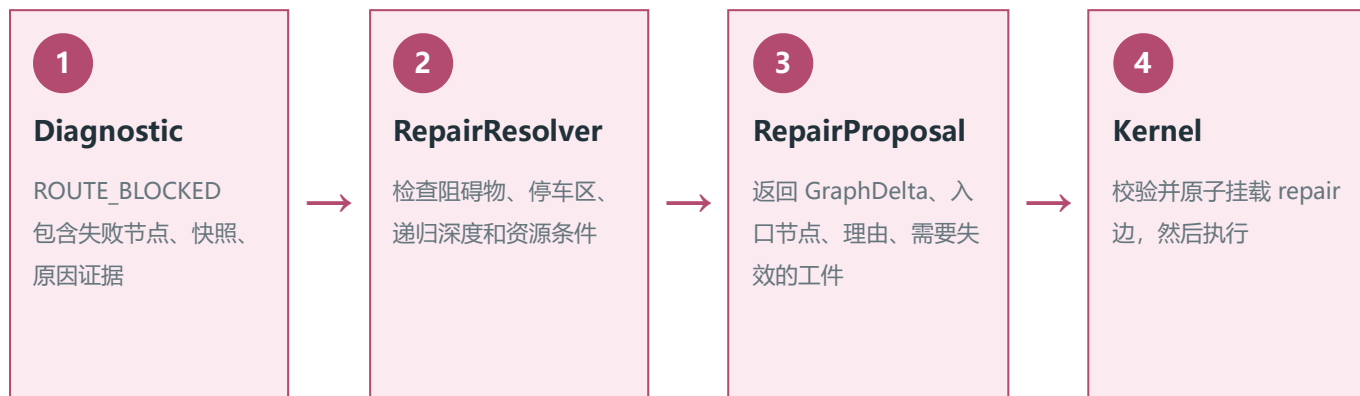
规则：算法可以证明“移动某障碍后会有路”，但不能在算法内部偷偷移动障碍。物理副作用必须成为任务树里显式的工作。

更直白地说

如果系统搬了东西，却没有在任务树里留下“谁搬的、为什么搬、搬到哪里、是否验证成功”，那它就无法可靠地解释和恢复。

故障如何从一句报错，变成一份可执行的修复方案？

Repair 不是一个隐藏的 catch 分支，而是一个结构化闭环。



Resolver 能做

- 读世界和工件。
- 分析故障原因。
- 提出修复子树。
- 说明理由和影响范围。

Resolver 不能做

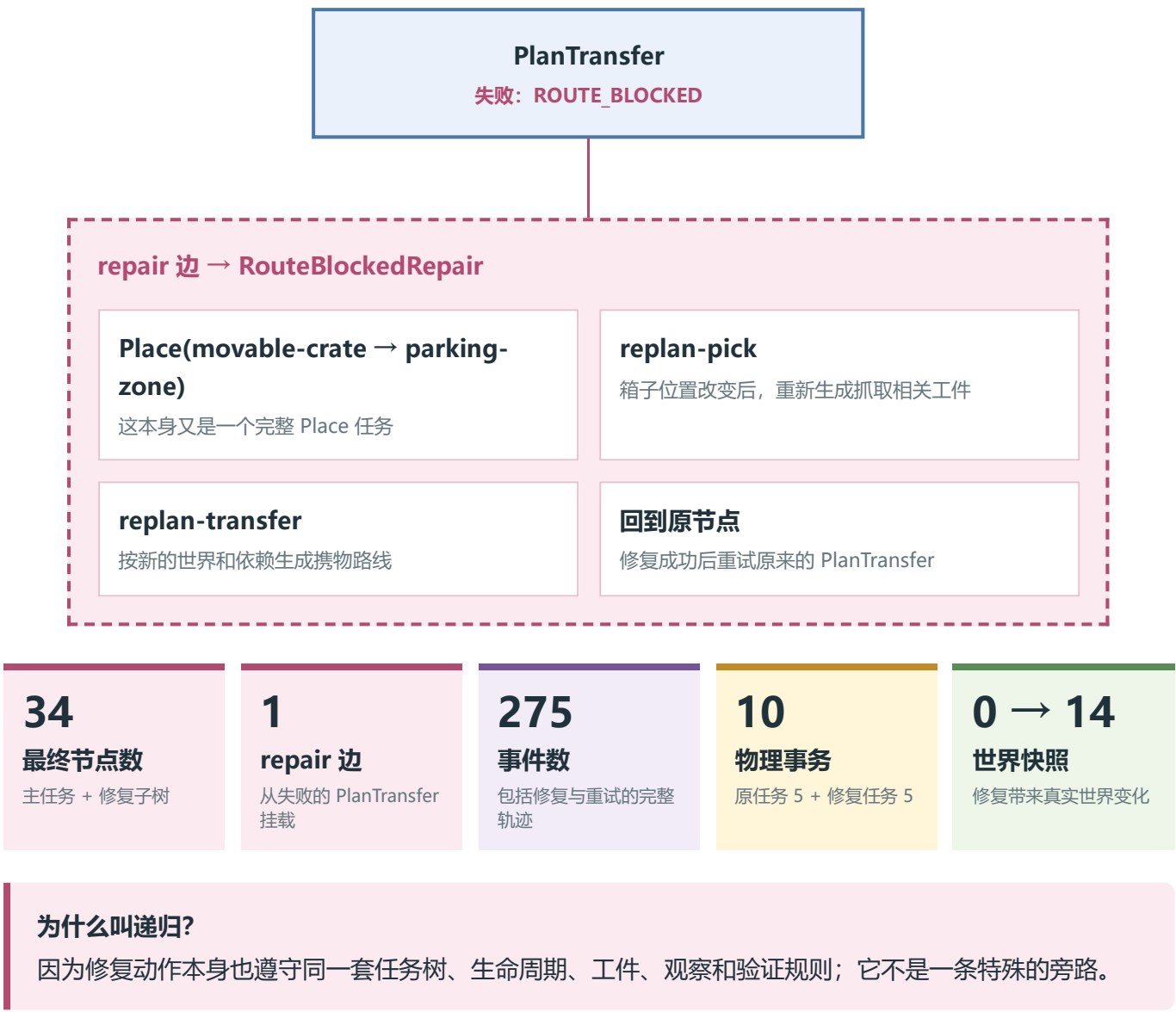
- 直接改 Store。
- 直接执行机器人动作。
- 自己宣布修复成功。
- 绕过 Kernel 挂边。

成功由谁判断？

不是 RepairResolver 说“我修好了”，而是 repair 子树执行后，由新的 Observation 和 Verifier 判断修复是否真的成功。

案例二：递归修复树到底长什么样？

修复任务不是漂浮在主流程外面的脚本，而是从失败节点下面显式挂出的一棵子树。



案例二：一次路线故障的详细执行过程

按时间顺序看，系统不是放弃原目标，而是先改变阻碍目标的现场条件。

1

PlanTransfer

在 world:0 规划携物路线，发现路线被 movable-crate 挡住。

产生 ROUTE_BLOCKED

2

反事实 A*

假设只移走这个箱子，重新计算后路线恢复；证明它是可修复 witness。

不是随便挑一个障碍物

3

挂载 repair

Kernel 校验 RepairProposal，并从失败的 PlanTransfer 挂一条 repair 边。

记录 rationale 与影响范围

4

递归 Place 箱子

对 movable-crate 执行完整的 Place：规划、抓取、携带、释放、验证。

世界推进到 world:7

5

刷新工件

箱子位置已经变化，原依赖可能过期；重新生成 PickPlan / TransferPlan。

新工件基于新快照

6

重试原节点

回到原来的 PlanTransfer，在新现场条件下重新执行。

不是换了用户目标

7

最终 Verify

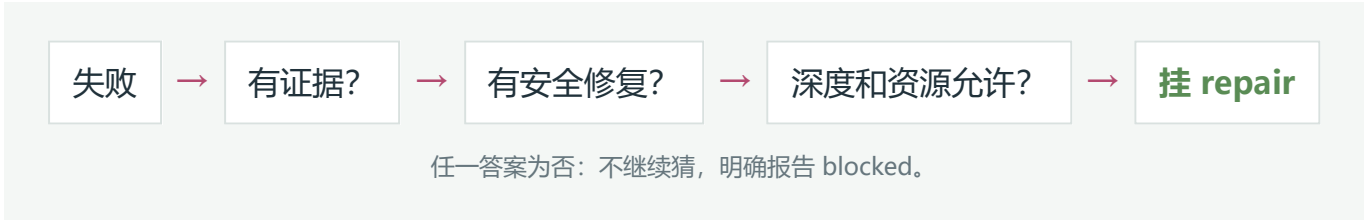
杯子到达 drop-zone，原 Place 根节点 succeeded。

世界最终到 world:14

递归修复会不会无限套娃？不会，系统有明确的收敛条件

“能修复”不等于“永远修复”。真正的工程系统必须知道什么时候停止。

递归深度上限 修复任务带有 repair_depth，超过上限就不能继续挂新的修复。	证据门槛 没有具体 movable witness、可行停车区和反事实路径证明，就不能提出搬障碍修复。
资源占用限制 如果机器人已经拿着杯子，不能再去抓第二个障碍物；应改为重新规划携物路线。	失败收敛 重复失败、无安全绕行、无可达停车区时，进入 blocked，交给人工或上层策略。



工程上的诚实

一个明确的 blocked，比系统不断尝试未知动作更有价值。它让人知道需要补充能力、改变现场，还是由人工接管。

连续多任务：树隔离，物理世界连续

这是整套架构很容易被误解的一点：每条指令有自己的树，但下一条指令看到的是上一条执行后的真实现场。



每个任务独立	跨任务共享
TaskTreeStore / Kernel / Inspector	WorldModel / HarnessRuntime / Backend
根节点、执行栈、事件和任务目标	机器人真实状态、实体位置、事务账本
防止两条根目标混在一棵树里	保证第二条指令看到第一条的结果

当前实现的准确描述

这是串行连续会话，不是已经完成的并行调度系统。若未来需要并行，必须另加资源调度、冲突仲裁和多任务一致性设计。

安全与治理：不是一句提示词，而是一串硬边界

物理动作的安全性来自多个层次共同作用，而不是寄希望于模型每次都理解对。

意图边界	LLM 只能提交规范化 TaskProgram
树边界	Kernel 唯一解释生命周期，Store 唯一权威状态
工件边界	计划带快照、依赖版本和安全策略指纹
执行边界	Runtime 统一做 freshness、guards、leases、checkpoint、事务
证据边界	每条命令后收 Observation，Verifier 只读快照
修复边界	失败变 Diagnostic，修复以 repair 子树显式出现

真实机器人和模拟器的差异

模拟器可以回滚内存；真实后端通常不能物理回滚。因此生产适配器应报告 `supports_rollback = False`，依靠 Observation reconciliation 对账，而不是假装动作被撤销。

高层设计原则：为什么这套架构值得做？

优势不在于模块数量更多，而在于把隐含行为变成可检查的契约。

单一所有权	一个 Store、一个 Kernel 写入者，减少状态冲突。	职责隔离	意图、规划、物理动作、验证分别由不同角色承担。
证据驱动	成功来自 Observation 和快照证据，不来自函数返回值。	工件不可变	历史不被偷偷修改，新现场发布新计划。
失败显式化	Diagnostic 和 repair 边让恢复进入正常工作流。	任务 / 世界分层	每条任务树可隔离，真实现场可连续复用。

管理层看到的价值	技术机制	可以观察的证据
风险可控	角色权限 + Runtime 唯一物理入口	ActionRequest、事务、Observation
故障可恢复	Diagnostic + repair subtree + 依赖重规划	repair 边、重试次数、最终状态
过程可解释	任务树、执行栈、事件日志、世界快照	127 / 275 个事件可回放
能力可替换	任务与算法、机器人通过协议隔离	替换 decomposer / backend 的契约测试

这套设计为什么值得这样拆？

这一页把前面看过的机制，翻译成你能用来判断系统好坏的理由。

少一点“玄学失败”

失败会落到具体节点、具体诊断和具体证据，减少“模型刚才不知道怎么了”。

更容易换能力

A*、IK、RRT、机器人后端可以替换，任务语义和控制面保持稳定。

少一点整条流程重做

障碍物只影响局部计划时，可以局部修复和局部重规划。

更容易做试点验收

可以定义成功率、恢复收敛率、人工介入率、过期计划拦截率和回放完整率。

任务成功率

最终 Verify 是否通过

恢复收敛率

发生故障后最终成功的比例

人工介入率

进入 blocked 后需要人处理的比例

回放完整率

任务、事件、工件、快照能否对齐

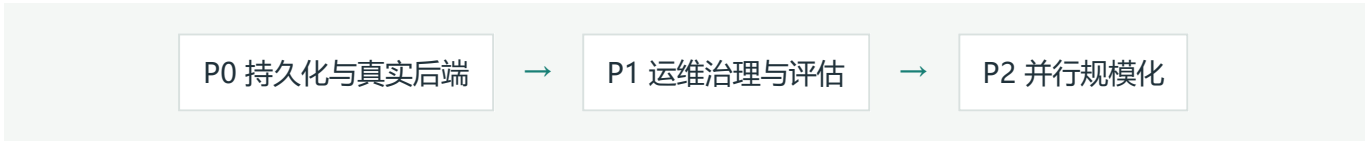
决策提醒

参考案例里的 16 / 34 节点、127 / 275 事件，是架构证据，不是对未来生产吞吐量或成功率的承诺。生产验收要用真实设备和真实任务重新测量。

当前参考实现，离生产系统还差什么？

架构契约已经明确，但参考实现仍然有清楚的边界。把边界讲出来，反而更专业。

当前参考实现	生产化需要补齐	优先级
Store 主要是内存实现	树、执行栈、事件、别名、工件需要同一事务持久化	P0
JSON 用于检查和导出	需要完整崩溃恢复协议和幂等重放	P0
确定性的平面双连杆模拟器	真实机器人适配、传感器噪声、控制周期和安全验收	P0
本地单操作员控制台	认证、权限、多操作员审计和部署隔离	P1
串行连续会话	如需并行，增加调度、资源仲裁和冲突解决	P1
保守网格 / 圆形碰撞模型	接入生产级碰撞库、附着载荷模型和在线监控	P1



未来增加新任务、新机器人、新算法，应该怎么接？

扩展点应该落在协议上，而不是把所有逻辑重新塞进 Kernel。

新增高层任务

- 增加已校验的 TaskProgram action。
- 只编译根节点。
- 注册 decomposer，返回 GraphDelta。
- 复用已有 system operation 和 physical skill。
- 补端到端测试和树形状断言。

新增机器人

- 实现 RobotBackend protocol。
- 把厂商 SDK 类型封装在 adapter 内。
- 把 telemetry 转成 Observation。
- 不把厂商私有状态暴露给 planner 或 verifier。
- 保留 ActionRequest / reconciliation 契约。

新增 MPC

- 实现 JointTrajectoryController。
- 注入 backend / controller adapter。
- 保持 ActionRequest 和 Observation 不变。
- 不要把 MPC 迭代变成任务树节点。
- 用安全边界和回放测试验收。

判断标准：新增能力应该改变“局部实现”，而不是破坏“任务树、工件、Runtime、Observation”的公共契约。

术语表：把技术词翻译成成人话

看到这些词时，先用右边的人话理解，再回头看左边的英文。

TaskProgram	模型提交的结构化任务意图，像标准订单。
TaskTree	会执行、会记录状态、会挂修复的任务树。
TaskTreeKernel	任务树的执行控制内核，决定下一步怎么推进。
TaskTreeStore	运行时树状态的唯一权威账本。
WorldModel	机器人和现场事实的版本化账本。
WorldSnapshot	某一时刻的不可变现场照片。
Artifact	基于某个快照生成的不可变施工图。
GraphDelta	受约束的树结构增量，不是随便改数据库。
Diagnostic	结构化失败原因，例如 ROUTE_BLOCKED。
RepairProposal	修复器提出的修复子树方案。
ActionRequest	物理技能生成的受限动作请求。
Observation	动作后观察到的真实证据。
reconciliation	无法物理回滚时，根据观察重新对账事实。
freshness	工件依赖是否仍与当前世界一致。

一行速记

TaskProgram 是订单，TaskTree 是施工计划，Artifact 是施工图，ActionRequest 是派工单，Observation 是现场照片。

如果你想继续往代码里看：模块分别在哪里？

这一页是查代码的地图，不要求一次记住；遇到名词时回来对照即可。

服务入口	<code>src/task_recursive_tree/integrations/gemini_er2/server.py</code> 浏览器 / HTTP / 连续会话入口
编译桥	<code>src/task_recursive_tree/integrations/gemini_er2/compiler.py</code> Harness DTO → Kernel 定义
执行桥	<code>src/task_recursive_tree/integrations/gemini_er2/executor.py</code> 装配 Store、Kernel、适配器
任务内核	<code>src/task_recursive_tree/task/kernel.py</code> 唯一生命周期解释器
权威 Store	<code>src/task_recursive_tree/task/store.py</code> 节点、运行态、边、栈、事件
节点模型	<code>src/task_recursive_tree/task/model.py</code> Node、Edge、GraphDelta
系统操作	<code>src/task_recursive_tree/integrations/gemini_er2/operations.py</code> 感知、选择、规划、验证
物理技能	<code>src/task_recursive_tree/integrations/gemini_er2/skill.py</code> Artifact → ActionRequest
物理运行时	<code>src/task_recursive_tree/integrations/gemini_er2/physical_runtime.py</code> 唯一后端执行边界
语义与修复	<code>src/task_recursive_tree/integrations/gemini_er2/semantics.py</code> / <code>repair.py</code> 验证、Diagnostic、RepairProposal

本 PDF 依据项目根目录的 ARCHITECTURE.md、README.md、中文架构说明和 .artifacts/ppt-examples 中的执行证据编写。参考数据用于解释架构，不代表生产 SLA。

最后只记住这张图：四句话讲完整套架构

如果你要向别人复述，按照下面四句讲，基本不会跑偏。

第一句：模型 LLM 负责理解人话、澄清目标、提交受约束的 TaskProgram。

第二句：控制 Kernel 负责解释任务树，Store 保存唯一权威状态，决定下一步怎么推进。

第三句：执行 系统基于世界快照生成不可变工件，物理技能把工件变成 ActionRequest，Runtime 才能碰机器人。

第四句：恢复 每条动作后收 Observation；失败变 Diagnostic，修复以 repair 子树进入同一套执行流程。

意图 → 任务树 → 工件 → 受控动作 → 观察证据 → 修复 / 完成

最终判断标准

不要问“模型看起来聪不聪明”，要问：任务是否可解释、动作是否受控、结果是否有证据、失败是否能收敛。

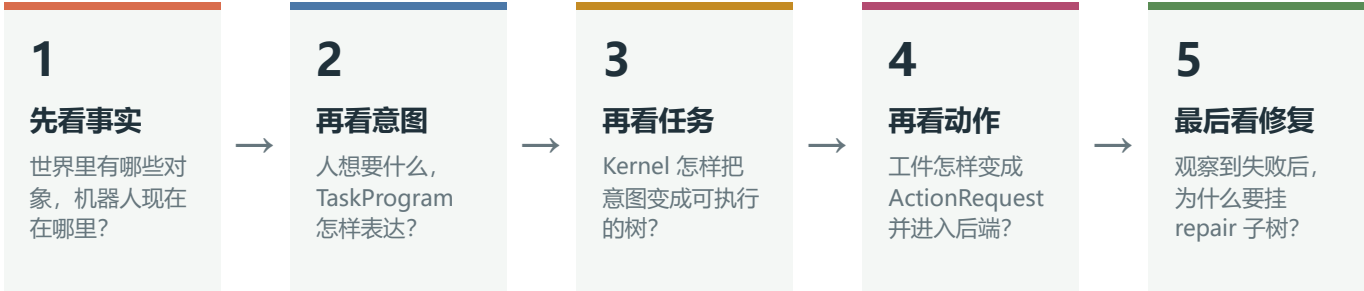
Task Recursive Tree Agent · 个人学习详解版

从这里开始：把它当成一门小课来学

这一组页面不是给别人汇报的讲稿，而是帮助你建立自己的架构直觉。

学习目标

读完后，你应该能回答六个问题：系统保存了什么？任务如何拆开？谁能写树？计划为什么会过期？动作后的事实从哪里来？失败如何变成修复任务？



读图口诀：事实 → 决定 → 计划 → 请求 → 观察 → 更新 → 再决定

不要先背英文

先把每个名字翻译成一句人话，再记英文名。

不要把计划当事实

计划只是“打算怎么做”，Observation 才是“做完后看见什么”。

不要把算法当任务

A*、IK、RRT 是工具内部的计算，不是用户要理解的业务步骤。

先只认识四种东西：目标、任务树、世界、工件

如果这四种东西没有分开，后面所有术语都会混在一起。

1. 目标 / Intent

“把红杯放到右边”

人真正想要的结果。它还没有说明每一步怎么做。

2. 任务树 / TaskTree

先规划、再抓取、再移动、再释放

把目标变成有顺序、有状态、有验证的工作清单。

3. 世界 / World

杯子在哪、箱子在哪、机器人是否拿着东西

系统对当前现场事实的版本化记录。

4. 工件 / Artifact

基于 world:0 生成的路线和姿态

把“怎么做”具体化，并记住它依赖哪个世界版本。



最容易犯的混淆

“我已经算出了路线” 不等于 “机器人已经走完了” ； “机器人收到了命令” 也不等于 “目标已经满足” 。架构把这些状态分开，正是为了不自欺。

你看到的東西	它回答的問題	它不回答的問題
TaskProgram	用戶想要什麼？	具體哪條軌跡最安全？
WorldSnapshot	現場現在是什麼樣？	下一步一定會成功嗎？
Artifact	按照哪個版本、怎样做？	動作後實際發生了什麼？
Observation	動作後觀察到了什麼？	是否應該直接修改任務樹？

组件为什么存在：从“会出错的地方”倒推设计

不要把组件看成一堆类名；每个组件都是为了堵住一种具体的错误。

如果没有它，容易发生什么	对应组件	它具体堵住的错误
模型直接发出任意机器人命令	TaskProgram + Compiler	把自由文本限制成可校验的任务意图。
多个模块同时修改任务树	TaskTreeStore	只允许一个 Kernel 持有写入能力。
任务做到一半不知道下一步	TaskTreeKernel	统一解释生命周期、子节点、验证和修复。
分解器偷偷把节点塞到别处	GraphDelta 校验	限制挂载范围、父子关系、无环和可达性。
路线还是旧现场的路线	ArtifactFreshness	比较快照、地图、实体和机器人版本。
系统操作绕过安全层直接碰机器人	HarnessRuntime	成为唯一物理执行入口。
命令成功但物体其实没抓住	Observation + Verifier	用动作后的事实判断，不信请求本身。
失败后模型临时发挥，无法回放	Diagnostic + Repair	把失败原因和修复工作显式放进树里。

把这页压缩成一句话： 每个边界都在回答“谁有权做这件事、依据什么事实、做完如何证明”。

学习方法
以后看到一个新组件，先问它要防哪一种错误。如果答不出来，说明你还没有理解它的设计动机，先不要急着记 API。

每个组件的高层原则：能做什么，也要知道不能做什么

架构的质量不只靠“它能完成任务”，还靠“它被禁止做什么”。

LLM / Agent

只提交意图，不提交副作用

可以理解、澄清、选择对象；不能写运行态树，不能直接调机器人。

Compiler

先校验，再进入执行世界

把 TaskProgram 编成根节点；不把任意外部 DTO 直接当权威运行态。

Store

单一事实来源

节点规格、运行态、边、执行栈和事件都由同一个 Store 保存。

Kernel

唯一生命周期解释器

只有它能推进状态、挂载 GraphDelta、处理失败和重试。

WorldModel

只记录可观察事实

世界版本由 Observation 推进；规划器不能把自己的猜测写成事实。

Artifact

不可变，依赖可检查

旧工件保留用于审计；世界改变时发布新工件，而不是偷偷改旧工件。

HarnessRuntime

副作用集中管理

做新鲜度检查、前置守卫、资源租约、事务和观察对账。

Verifier / Repair

结果先证据，修复再入树

Verifier 不调用规划器；RepairResolver 只能提出方案，不能自己改树。

允许的方向

任务树读世界 → 系统操作生成工件 → 技能构造请求 → Runtime 执行 → Observation 更新世界

禁止的捷径

模型直接调机器人、A* 偷偷搬障碍、技能直接改状态、验证器自己重规划。

世界状态先讲清：系统到底在记录什么？

后面的规划、守卫、验证和修复，都只能基于某个版本的世界状态。

WorldSnapshot: 某一个版本的现场事实

它像一张“带编号的照片”，不是模型随口猜出来的记忆

版本外壳

revision = 7 · snapshot_ref = world:7 · frame_graph_revision = 0

RobotState

base_pose
joints
gripper_open
held_object_id
机器人当前状态

entities

物体: cup-red
区域: drop-zone
障碍: movable-crate
每个实体有自己的 version

GridMap

width / height
resolution / origin
static_occupied
路线规划使用的地图事实

世界模型 WorldModel

像一个持续更新的“现场账本”。它接收 Observation，并在内存中维护当前状态。

世界快照 WorldSnapshot

像从账本上取出的只读照片。规划器和 Verifier 读取它，但不能直接改它。

版本 revision

每次 ingest 一次观察就推进一个世界版本；例如 world:0、world:1、world:7。

事实与计划

“箱子在 (4.5, 3.5)” 是事实；“从这里走到那里” 是计划，两者不能混写。

参考实现的边界

当前参考实现的 WorldModel 和 TaskTreeStore 都主要在内存里；JSON 导出方便检查和回放，不等于已经具备完整的崩溃恢复协议。

WorldSnapshot 的具体编码：看一张完整的“现场照片”

下面是为了阅读整理的 JSON 形态；真实 Python dataclass 导出会把元组、枚举和集合转换成 JSON 数组或字符串。

```
{
  "snapshot_ref": "world:0",
  "revision": 0,
  "frame_graph_revision": 0,
  "robot": {
    "base_pose": {"x": 1.5, "y": 1.5, "z": 0.0, "yaw": 0.0, "frame": "world"},
    "joints": [0.0, 0.0],
    "gripper_open": true,
    "held_object_id": null,
    "state_epoch": 0
  },
  "entities": {
    "cup-red": {
      "kind": "object", "pose": {"x": 2.5, "y": 2.5, "z": 0.0, "yaw": 0.0, "frame": "world"},
      "radius": 0.16, "tags": ["cup", "red"],
      "properties": {"movable": true}, "version": 0
    },
    "drop-zone": {
      "kind": "region", "pose": {"x": 7.5, "y": 5.5, "z": 0.0, "yaw": 0.0, "frame": "world"},
      "radius": 0.45, "tags": ["destination"],
      "properties": {"acceptance_radius": 0.3, "placement_offsets": [[0.0, 0.0]]},
      "version": 0
    }
  },
  "grid": {
    "width": 10, "height": 8, "resolution": 1.0,
    "origin_x": 0.0, "origin_y": 0.0,
    "static_occupied": [[4, 0], [4, 1], [4, 2], [4, 4]]
  }
}
```

字段	你可以把它理解成	它为什么重要
snapshot_ref	这张照片的编号	工件和证据都能说清自己基于哪一版现场。
revision	世界版本计数器	判断“现场是否更新过”。
robot	机器人自己的状态	抓取、移动、释放的前提都要依赖它。
entities	物体、区域、障碍物	选择对象、碰撞判断和目标验证都从这里读。
grid	路线规划使用的地图	A* 需要知道哪些格子可走。

EntityState：一个物体的状态到底包括哪些属性？

先把一个杯子拆开看；理解了一个实体，理解区域和障碍就容易了。

```
{
  "entity_id": "cup-red",
  "kind": "object",
  "pose": {
    "x": 2.5,
    "y": 2.5,
    "z": 0.0,
    "yaw": 0.0,
    "frame": "world"
  },
  "radius": 0.16,
  "tags": ["cup", "red"],
  "properties": {
    "movable": true
  },
  "version": 0
}
```

字段	白话解释	在杯子任务中怎么用
entity_id	稳定身份证，不是显示名称	任务引用 cup-red；即使展示文案改变，引用仍然稳定。
kind	实体大类	object 表示物体；region 表示目标或停放区域。
pose	位置、方向、坐标系	规划器知道杯子在哪里；frame 防止把不同坐标系混在一起。
radius	用于碰撞和空间近似的尺寸	杯子半径影响可达性、占用格子和载荷包络。
tags	语义标签集合	可用 cup、red 等标签做选择，不必只靠名字猜。
properties	领域属性字典	movable 表示可移动；nav_obstacle 表示会挡导航。
version	这个实体自己的变化次数	杯子被放下后版本增加，旧计划就可能不再新鲜。

为什么不把所有东西塞进一个 description 字符串？

因为规划和验证需要可计算的字段。字符串适合给人看，结构化属性才适合做前置条件、碰撞判断和版本比较。

物体、区域、障碍物：看起来都叫实体，实际用途不同

它们共享 EntityState 的外壳，但通过 kind、tags 和 properties 表达不同语义。

object · 物体

```
{
  "entity_id": "movable-crate",
  "kind": "object",
  "radius": 0.35,
  "tags": ["crate"],
  "properties": {
    "movable": true,
    "nav_obstacle": true
  }
}
```

可以被抓取和搬运；同时会占用导航空间。

region · 区域

```
{
  "entity_id": "drop-zone",
  "kind": "region",
  "radius": 0.45,
  "tags": ["destination"],
  "properties": {
    "acceptance_radius": 0.3,
    "placement_offsets": [[0.0, 0.0]]
  }
}
```

它不是要被搬走的东西，而是判断“放得够不够近”的参考区域。

robot · 为什么单独编码

```
{
  "base_pose": {"x": 1.5, "y": 1.5, "yaw": 0.0},
  "joints": [0.0, 0.0],
  "gripper_open": true,
  "held_object_id": null,
  "state_epoch": 0
}
```

机器人有可驱动关节、夹爪和资源占用，不适合伪装成普通实体。

问题	读哪个字段	例子
“它是不是我想找杯子？”	kind + tags + selector	kind=object, tags 包含 cup、red。
“它会不会挡路？”	properties.nav_obstacle	movable-crate=true。
“它能不能被搬走？”	properties.movable	箱子可移动，静态墙通常不可移动。
“放到哪里算成功？”	region.properties	acceptance_radius=0.3。

机器人状态与版本：为什么 state_epoch 不能省？

世界版本告诉你“现场更新过几次”，机器人状态纪元告诉你“机器人自身是否变过”。



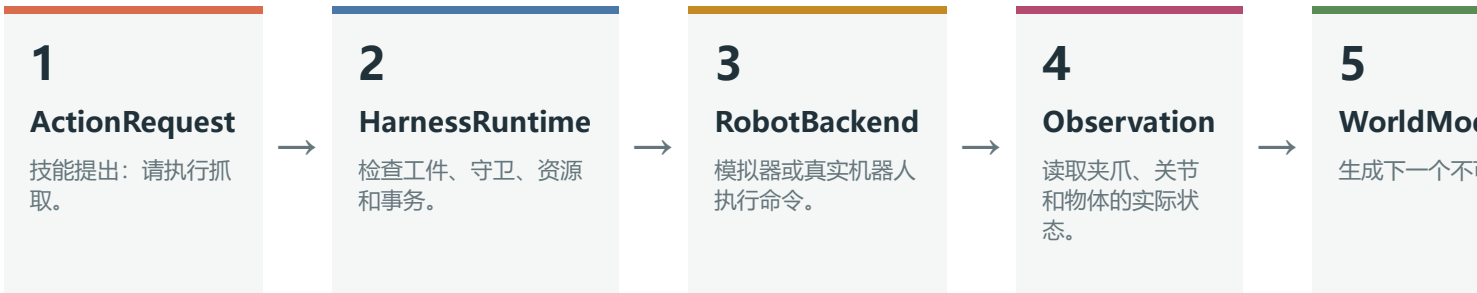
属性	记录什么	如果忽略会怎样
base_pose	移动底盘的位置和朝向	导航计划的起点可能已经不对。
joints	机械臂当前关节角	旧的抓取轨迹可能无法连续接上。
gripper_open	夹爪是否张开	无法判断能否抓取或释放。
held_object_id	当前是否持有某物	系统可能错误地尝试抓第二个物体。
state_epoch	机器人状态变化的纪元	工件无法检测“轨迹起点已变”。

参考模型边界

当前项目里的机器人是确定性的平面双连杆参考模拟器。这里的字段契约可保留到真实机器人，但真实控制器、噪声和在线安全监控仍需另行接入。

Observation 如何推进世界：动作之后才有新事实

把“请求”和“事实”分开，是这套架构最重要的学习点之一。



动作前: world:1

```
robot.gripper_open = true
robot.held_object_id = null
cup-red.version = 0
```

收到 Observation →

动作后: world:2

```
robot.gripper_open = false
robot.held_object_id = "cup-red"
robot.state_epoch = 2
cup-red.version = 0
```

阶段	它是什么	谁可以改变它
请求	ActionRequest: 想让后端做什么	PhysicalSkill 构造, Runtime 接收。
执行	Backend: 实际尝试做什么	HarnessRuntime 唯一调用入口。
观察	Observation: 动作后看见什么	Backend 提供, WorldModel 接收。
结论	Verifier: 后置条件是否满足	只读快照, 返回结果和证据。

关键句

命令返回成功，只说明命令被执行流程接受或完成；只有 Observation 加上 Verifier，才能说明杯子真的被夹住、真的到达目标。

任务不是一句话：TaskProgram 怎样表达 “我要做什么”

LLM 的输出要先变成一个受约束的中间表示，才允许进入任务树；选择器的完整字段放在下一页。



```
{
  "program_id": "demo-place",
  "action": "place",
  "arguments": {
    "object_selector": {
      "entity_kind": "object",
      "relation": "exact",
      "entity_id": "cup-red",
      "required_tags": ["cup", "red"]
    },
    "destination_selector": {
      "entity_kind": "region",
      "relation": "exact",
      "entity_id": "drop-zone",
      "required_tags": ["destination"]
    }
  }
}
```

字段	白话意义	设计动机
program_id	这份任务意图的身份证	让根节点、日志和会话历史能对上。
action	高层动作类型	当前参考实现支持 place，其他动作先拒绝。
object_selector	怎样找被操作的物体	用结构化选择代替模型猜名字。
destination_selector	怎样找目标区域	目标也要明确绑定，不能只说“右边”。
cardinality / tie_policy	必须选几个、同分时如何决胜	防止一次选择多个或结果不稳定。

TaskProgram 里的选择器：系统怎样确认“到底是哪一个对象”？

任务里的对象不能只靠一句描述；选择器把语义、空间和决胜规则写成可检查的结构。

```
{
  "entity_kind": "object",
  "relation": "exact",
  "entity_id": "cup-red",
  "required_tags": ["cup", "red"],
  "region_id": null,
  "reference_entity_id": null,
  "reference": "robot",
  "frame": "world",
  "metric": "euclidean",
  "cardinality": "one",
  "tie_policy": "stable_id"
}
```

选择器字段	它在说什么	为什么不能省
entity_kind	我要找 object 还是 region	避免把目标区域当成要搬的物体。
relation	exact、nearest、leftmost 等选择关系	把“怎么选”从模型猜测变成规则。
entity_id / required_tags	精确 ID 或语义标签	既能稳定引用，也能表达类别。
reference / frame	相对于谁、在哪个坐标系	防止“右边”“附近”失去参照。
metric / cardinality	如何计算距离、选几个	保证选择结果可重复、不会偷偷选多个。
tie_policy	同分时谁优先	让同一现场多次运行得到稳定结果。

SpatialSelector描述选择规则

→

SelectorEngine读取 WorldSnapshot 并筛选

→

BindingArtifact记录最终绑定和证据

```
{
  "artifact_id": "binding-f74f98cb32e14750bae35db65b7744d0",
  "object_id": "cup-red",
  "destination_id": "drop-zone",
  "object_candidates": ["cup-red"],
  "destination_candidates": ["drop-zone"],
  "snapshot_ref": "world:0",
  "grounding_evidence": ["exact id and required tags matched"]
}
```

TaskNodeSpec: 任务树节点的“身份证和工作说明书”

Spec 描述节点本来是什么；它不记录这个节点已经跑了几次。

```
{
  "node_id": "program/demo-place",
  "task_type": "Place",
  "operation_kind": "decomposer",
  "control_kind": "sequence",
  "origin": "program",
  "parameters": {
    "scope": "demo-place",
    "object_selector": "exact cup-red",
    "destination_selector": "exact drop-zone",
    "repair_depth": 0
  },
  "preconditions": [],
  "postconditions": [],
  "max_attempts": 1,
  "max_repairs": 0,
  "execution_policy": "skip_if_goal_satisfied"
}
```

字段	它回答的问题	Place 示例
node_id	这个节点在树中的稳定位置？	program/demo-place/plan-pick
task_type	业务上要做哪件事？	Place、PlanPick、ExecuteGrasp。
operation_kind	它属于哪种权限角色？	decomposer、system 或 physical。
control_kind	它怎样控制子节点？	leaf、sequence、selector。
origin	它从哪里产生？	program、compiler、decomposer 或 repair。
parameters	它执行时需要哪些参数？	对象、目标、scope、repair_depth。
pre/postconditions	开始前和结束后要满足什么？	object_held、placed_in_region 等。
limits / policy	最多重试几次、目标已满足时怎样处理？	防止无限循环和重复副作用。

读一个节点的方法： 先看它是什么角色，再看参数；然后看它前置、后置和上限。不要只看 task_type，因为同名任务在不同角色下可能拥有不同权限。

TaskNodeRuntime： 同一个节点现在跑到哪一步了？

运行态会变，任务定义尽量不变；这就是为什么两者要分开。

```
{
  "status": "running",
  "phase": "preconditions",
  "attempts": 1,
  "repairs": 0,
  "expanded": true,
  "output_artifacts": [
    "nav-dfef826f0b0c4b1faa566d1447acadd5",
    "pick-efbdc179e47a459d94fe036166172764"
  ],
  "last_diagnostic": null,
  "started_at": 1725000000.0,
  "finished_at": null,
  "adapter_state": {
    "active_request_id": "tree:demo-place:...:attempt:1"
  }
}
```



字段	它记录什么	学习时怎么理解
status	pending / running / succeeded / failed / blocked / cancelled	节点的最终或当前结果。
phase	enter、preconditions、execute、verify、repair 等	这个节点内部生命周期走到哪一站。
attempts	已经尝试执行几次	区分首次失败和重试失败。
repairs	已经挂过几次修复	防止修复无限递归。
expanded	是否已经展开过子节点	复合节点不能每次 tick 都重复加孩子。
output_artifacts	这个节点发布了哪些工件	后面的物理节点通过 scope 找到计划。
last_diagnostic	最近一次结构化失败	让失败不只是一个字符串。

为什么一定要把 Spec 和 Runtime 分离？

把它想成“施工图”和“施工记录”：图纸不能因为今天下雨就被改写。



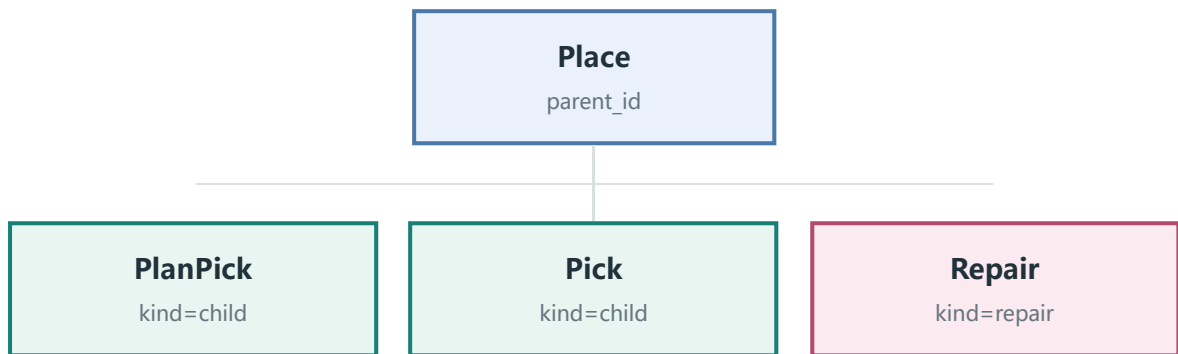
时刻	Spec 保持什么	Runtime 发生什么
刚编译	Place 是 sequence，参数是 cup-red → drop-zone	status=pending，attempts=0。
开始执行	任务定义不变	status=running，phase=enter。
发布计划	PlanPick 仍然是 PlanPick	output_artifacts 写入 pick / nav 引用。
路线失败	原节点仍然代表 PlanTransfer	last_diagnostic=ROUTE_BLOCKED，repairs=1。
修复成功	原任务仍可被重新解释	repair 子树 succeeded，原节点再次尝试。

如果混在一起会怎样？

为了重试而直接改 task_type、参数和历史状态，最后你无法判断：原来要做什么？后来为什么变了？哪一次动作对应哪份计划？分离让定义和执行记录各自清楚。

TaskEdge 和 GraphDelta：树不是靠“改一大坨 JSON”长出来的

分解器每次只提交一个受约束的结构增量，Kernel 验证后才交给 Store。



```

// 普通 child 边
{
  "parent_id": "program/demo-place",
  "child_id": "program/demo-place/pick",
  "kind": "child",
  "order": 3
}

// 失败节点挂出的 repair 边
{
  "parent_id": "program/demo-place/plan-transfer",
  "child_id": "program/demo-place/plan-transfer/repair-0/route-blocked",
  "kind": "repair",
  "order": 0
}
  
```

GraphDelta 带来什么

只描述新增节点和新增边；外部组件没有机会直接重写整棵运行时树。

Kernel 检查什么

不能自环、不能重复父节点、不能成环、不能脱离根节点、repair 必须从失败节点挂出。

Store 做什么

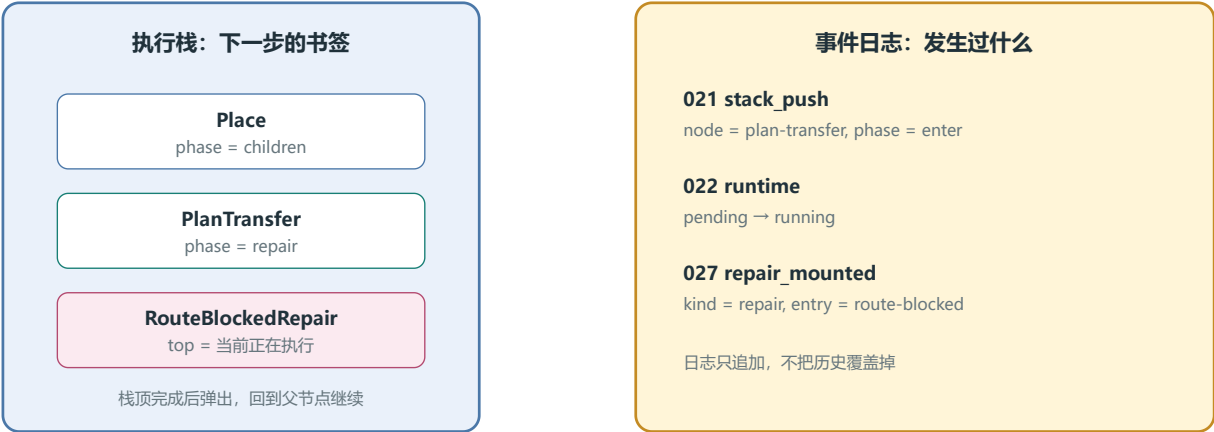
通过唯一 writer capability 原子应用增量，并追加 graph_delta 事件。

为什么叫“增量”

因为系统需要知道树是怎样一步步长出来的。每个 GraphDelta 都能对应到某个分解、修复或编译动作，审计时可以回放。

执行栈和事件日志：一个管“下一步”，一个管“已经发生过什么”

这两个对象常被混为一谈；把它们分清，你就能读懂 Kernel 的运行过程。



对象	像什么	包含什么	用途
ExecutionFrame	书签 / 调用栈帧	node_id、phase、next_child_index、active_child_id、诊断	暂停后知道从哪里继续。
Execution stack	正在展开的路径	从根到当前节点的一串 frame	表达当前控制位置。
TaskEvent	黑匣子流水账	sequence、event_type、node_id、data	解释过去发生过什么。
事件日志	只追加的历史	graph_delta、runtime、stack_push/pop、repair_mounted	回放、调试和审计。

一个简单比喻

执行栈像你手里夹着的书签：告诉你现在读到哪；事件日志像读书笔记：记录你之前翻过哪些页、在哪里停过、为什么回头。

Artifact：它不是一条孤零零的路线，而是一份带版本的施工图

如果只保存 path，不保存它基于什么现场算出来，系统就无法判断它还能不能用。

```
{
  "artifact_id": "transfer-58e75ad6f16f4a4bbb247fec6d4f7fdb",
  "kind": "transfer_plan",
  "metadata": {
    "snapshot_ref": "world:0",
    "dependency_versions": [
      ["entity:drop-zone", 0],
      ["frame_graph", 0],
      ["map", 0],
      ["nav_occupancy:cup-red", 10334876803476250410]
    ],
    "frame_graph_revision": 0,
    "robot_state_epoch": 0,
    "robot_model_version": "planar-2link-v1",
    "collision_model_version": "grid-circle-v1",
    "payload_transform_hash": "cup-red:0.160000",
    "assumptions": ["gripper is empty at planning time"],
    "random_seed": 0
  },
  "object_id": "cup-red",
  "destination_id": "drop-zone",
  "navigation_plan_ref": "nav-9cc2bc1deef4bd59f89a58886e5499d",
  "placement_pose": {"x": 7.5, "y": 5.5, "z": 0.0, "yaw": 0.0, "frame": "world"}
}
```

元数据	它记住什么	用来拦截什么问题
source snapshot	从 world:0 哪张照片算出来	不把旧现场计划当新现场。
dependency_versions	地图、实体、机器人、占用指纹的版本	障碍物或机器人姿态变化。
model versions	机器人和碰撞模型版本	算法/模型升级后继续误用旧计划。
payload_transform_hash	载荷几何身份	拿了不同尺寸的物体仍沿用旧轨迹。
assumptions / seed	计算时的前提和随机种子	复现、审计和解释规划结果。

新鲜度判断： 当前快照的依赖版本全部匹配 → 工件可消费；任意关键依赖不匹配 → 工件过期，发布新版本。

ActionRequest 和 Observation：请求不是结果

下面把“执行抓取”从规划工件一路翻译到机器人接口。

PhysicalSkill 构造请求

```
{
  "request_id": "tree:demo-place:program/demo-
place/pick/grasp:attempt:1",
  "action_name": "ExecuteGrasp",
  "commands": [
    {"type": "ArmPathCommand", "path":
"pick.approach_path"},
    {"type": "GripperCommand", "close": true,
"object_id": "cup-red"}
  ],
  "resources": ["arm", "gripper"],
  "artifact_refs": ["pick-
efbdc179e47a459d94fe036166172764"],
  "preconditions": [
    {"name": "base_near", "tolerance": 0.2},
    {"name": "joints_near", "tolerance": 0.00001}
  ]
}
```

Backend 执行后返回观察

```
{
  "robot": {
    "base_pose": {"x": 2.5, "y": 2.5, "yaw": 0.0},
    "joints": [0.486695, -0.973390],
    "gripper_open": false,
    "held_object_id": "cup-red",
    "state_epoch": 2
  },
  "entities": ["cup-red at (2.5, 2.5)"],
  "frame_graph_revision": 0
}
```

ActionRequest 字段	作用
request_id	把一次动作和节点、attempt、事务记录关联起来。
commands	真正要交给 backend 的类型化命令序列。
resources	申请 arm、base、gripper 等资源租约。
artifact_refs	声明动作依赖哪些不可变工件。
preconditions	Runtime 在当前快照上先验证的守卫。

你要记住的顺序

Skill 只能构造请求；HarnessRuntime 才检查新鲜度、守卫、资源和事务；Backend 执行后必须观察；WorldModel 再生成新版本。

Diagnostic 和 RepairProposal: 失败如何变成结构化工作

一个错误字符串只能告诉你“坏了”；结构化诊断要告诉系统“哪里坏、依据什么、能不能修”。

Diagnostic

```
{
  "code": "ROUTE_BLOCKED",
  "message": "candidate obstacle blocks route",
  "details": {
    "failed_node_id": "program/demo-place/plan-transfer",
    "snapshot_ref": "world:0",
    "witness": "movable-crate",
    "counterfactual_route": true
  },
  "retryable": true
}
```

它描述已经发生的失败，并附上证据和是否值得尝试恢复。

RepairProposal

```
{
  "entry_node_id": ".../repair-0/route-blocked",
  "rationale": "relocate movable-crate to parking-zone, then refresh PickPlan",
  "kind": "repair",
  "invalidates_artifacts": [],
  "delta": {
    "new_nodes": ["RouteBlockedRepair", "Place(crate, parking-zone)", "PlanPick"],
    "new_edges": ["failed-node --repair--> entry"]
  }
}
```

它只是“建议挂什么修复子树”；真正挂载仍由 Kernel 完成。

失败类型	说明	下一步通常是什么
ROUTE_BLOCKED	路线被具体障碍物挡住	验证可移动 witness，执行递归搬移修复。
STALE_PLAN	工件依赖版本不匹配	重新规划，不重复执行旧工件。
HOLD_LOST	观察到物体不再被持有	根据现场事实重新对账或进入人工恢复。
OUTCOME_UNKNOWN	证据不足，无法确认结果	先观察/人工确认，不能直接宣称成功。
GUARD_FAILED	动作前置条件不成立	不执行副作用，重新定位或修复前置条件。

两条边界

Verifier 负责判断和给证据，不负责规划；RepairResolver 负责提出 GraphDelta，不负责直接写 Store。这样修复本身也必须被 Kernel 管住。

完整例子一：正常 Place 从意图走到完成

把前面所有对象串在一起，先看没有故障时的最短路径。

输入 Place(cup-red → drop-zone)	结果 succeeded	范围 world:0 → world:7
---	------------------------	--------------------------------

序号	树节点 / 组件	产生或读取的对象	发生了什么
1	LLM / Planner Adapter	TaskProgram	把人话规范化为 place + 两个精确选择器。
2	Compiler	TaskNodeSpec(root)	只编译出 program/demo-place 根节点。
3	Kernel + Decomposer	GraphDelta	展开 Place，挂 7 个直接 child。
4	ResolveAndInspect	BindingArtifact	确认 cup-red 和 drop-zone 存在且绑定稳定。
5	PlanPick	NavigationPlan + PickPlan	基于 world:0 计算抓取位置和机械臂路径。
6	PlanTransfer	NavigationPlan + TransferPlan	提前检查持物运输路线。
7	Pick	5 个物理事务中的前 2 个	到抓取位、闭合夹爪，Observation 确认持有。
8	TransferHeld	第 3、4 个物理事务	切换运输姿态并带杯移动到目标附近。
9	Release + Verify	第 5 个物理事务 + 新快照	释放杯子，验证它落在接受半径内。

最终节点 16 业务步骤和验证节点	child 边 15 树结构关系	repair 边 0 没有发生故障	事件 127 可回放执行过程	物理事务 5 每个都有观察
---------------------------------------	--------------------------------------	---------------------------------------	------------------------------------	-----------------------------------

读法
先看树的业务顺序，再看每个物理节点的请求和观察；不要把 127 个事件误解成 127 次机器人动作。

完整例子二：路线阻塞时，任务树怎样递归长出来？

这是最能体现架构价值的例子：系统没有偷偷绕过失败，而是把修复本身变成一棵可执行子树。



1. 先发现

PlanTransfer 在 world:0 发现 movable-crate 挡路，并用反事实 A* 证明“移走这个具体箱子后会恢复路线”。

2. 再提案

Diagnostic=ROUTE_BLOCKED; RepairResolver 提出 RouteBlockedRepair，不直接改树。

3. 挂修复

Kernel 从失败节点挂一条 repair 边，入口是 repair-0/route-blocked。

4. 递归执行

修复子树内部再次执行 Place(movable-crate → parking-zone)，所以它也有解析、规划、抓取、运输、释放、验证。

5. 刷新再试

箱子到 parking-zone 后得到 world:7，重新发布依赖新世界的 PickPlan / TransferPlan，重试原任务。

```
Place(cup-red -> drop-zone)
├─ ResolveAndInspect
├─ PlanPick
├─ PlanTransfer
├─ RouteBlockedRepair [repair]
│   └─ Place(movable-crate -> parking-zone)
│       ├── ResolveAndInspect
│       ├── PlanPick
│       ├── PlanTransfer
│       ├── Pick
│       ├── TransferHeld
│       ├── Release
│       └─ VerifyPlaceGoal
└─ PlanPick
├─ Pick
├─ TransferHeld
├─ Release
└─ VerifyPlaceGoal
```

把故障案例的数字和连续任务放在一起看

树可以隔离，物理世界不能凭空重置；这两个事实需要同时记住。

最终节点
34
原任务 + 修复子树

总边数
33
其中 1 条 repair 边

repair 边
1
从 PlanTransfer 挂出

事件
275
包含挂修复和两段执行

物理事务
10
先搬箱子 5 次，再搬杯子 5 次

Task 0001
Place(movable-crate → parking-zone)
root = program/task-0001
5 次物理事务：world:0 → world:7
任务树执行完后，箱子真的留在 parking-zone。

同一个物理世界继续
world:7 →

Task 0002
Place(cup-red → drop-zone)
root = program/task-0002
5 次物理事务：world:7 → world:14
新任务树干干净，但读取的是上一个任务留下的现场。

每个任务新建	跨任务复用	为什么这样做
TaskTreeStore / Kernel / Inspector	WorldModel / HarnessRuntime	树之间不混节点，现场却保持连续。
执行栈 / 节点运行态 / 事件	RobotBackend / 事务账本	任务历史隔离，真实资源和动作历史连续。
root = program/task-0001 或 0002	同一场景状态	可以单独查看每棵树，也能解释世界如何演进。

当前实现的边界
连续会话目前是串行提交和串行执行，不要把它描述成已经完成了并行调度。若未来并行，需要新增资源仲裁、冲突解决和调度层。

学完后的自测：你是否真的看懂了？

不用背所有类名；能用下面的问题解释系统，就说明主线已经掌握。

01 为什么 LLM 不能直接调机器人？

因为模型输出要先变成受约束的 TaskProgram，副作用必须经过 Runtime。

02 谁是运行时任务树的唯一权威？

TaskTreeStore 保存状态，TaskTreeKernel 是唯一生命周期解释器和唯一写入者。

03 世界状态怎样推进？

物理命令后产生 Observation，WorldModel.ingest 生成新的 WorldSnapshot 版本。

04 为什么旧计划不能直接继续用？

Artifact 记录快照和依赖版本；当前世界不匹配时，计划过期。

05 为什么失败要挂 repair 边？

让修复动作成为可见、可验证、可回放的任务，而不是隐藏副作用。

06 为什么 A*、IK、RRT 不是任务节点？

它们是能力内部算法；任务树只记录对业务有意义的操作和验证。

自测标准

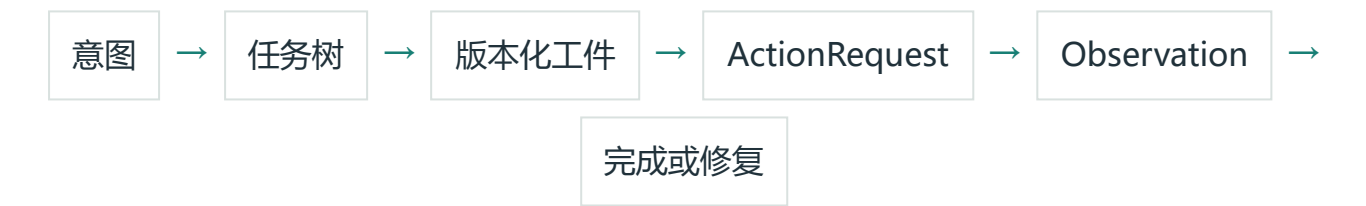
如果你能用自已的话解释上面六个问题，说明你已经抓住主线；如果某一题说不清，就回到对应的课程页查对象和例子。

Task Recursive Tree Agent · 个人学习详解版

一张图复习：从人话到可验证的物理结果

把整套架构重新压缩成六个问题，你以后读代码就按这个顺序定位。

人话	我想把哪个对象放到哪里？这会变成什么样的 TaskProgram？
事实	当前 world:N 里对象、机器人、地图和版本号是什么状态？
控制	Kernel 按什么树、什么 phase、什么边继续推进？谁有权写入 Store？
计划	这个 Artifact 基于哪个快照、哪些依赖和哪些模型版本？
动作	请求经过哪些前置守卫、资源租约、事务和唯一后端入口？
证据	Observation 证明了什么？失败后修复是否仍然以 repair 子树回到同一套执行器？



最后一句话

这套架构的核心不是让模型“更会想”，而是让每一次决定都有来源、每一次动作都有边界、每一个结果都有证据、每一次失败都有可见的下一步。

Task Recursive Tree Agent · 个人学习详解版